
Q1.

Given that A is a Boolean variable, evaluate and then simplify the following expressions: [Storey, Electronics: A Systems Approach, 5th Ed., Pearson]

- $A \cdot 1$
- $A \cdot \overline{A}$
- $1 + A$
- $A + \overline{A}$
- $1 \cdot 0$
- $1 + 0$

Solution:

- $A \cdot 1 = A$ (Identity)
- $A \cdot \overline{A} = 0$ (And-Complement)
- $1 + A = 1$ (Annihilation)
- $A + \overline{A} = 1$ (Or-Complement)
- $1 \cdot 0 = 0$ (Annihilation)
- $1 + 0 = 1$ (Identity)

Q2.

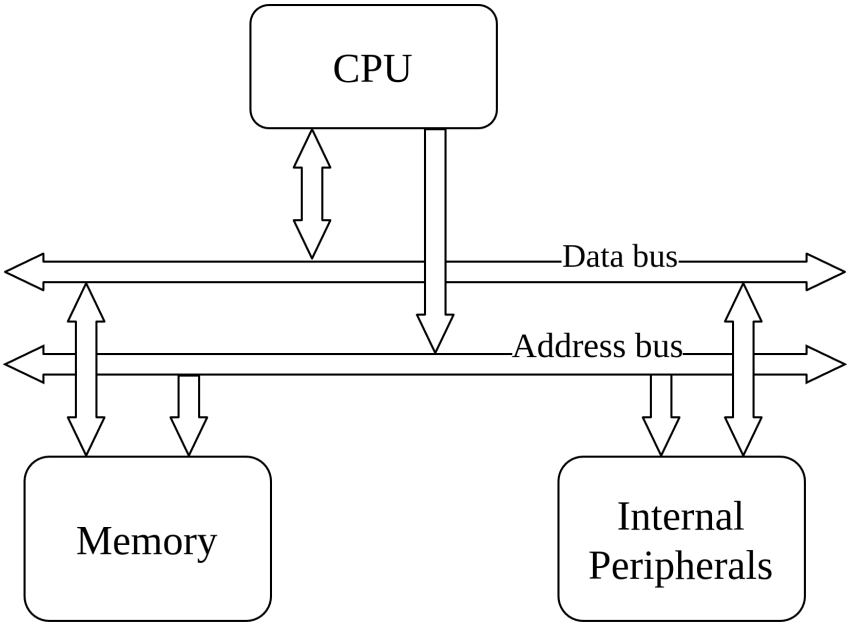
You want to change the setting of a register in one of the internal peripherals within your ATMega32u2.

(a) First you want to change just a single bit in the register. Explain why you cannot simply write to a single bit of a register.

Solution: A register is built in hardware (i.e. physical transistors) such that they must be read and written at the same time. Therefore the entire register/byte must be written at the same time.

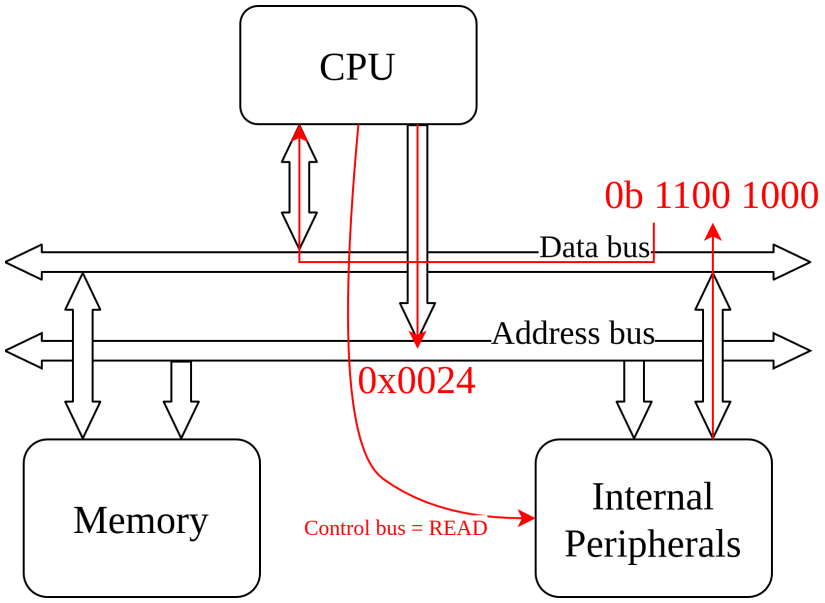
(b) How many bits are read/modified by default by each operation within the ATMega32u2?

Solution: The ATMega32u2 is an 8-bit microcontroller, which means its word length is 8. Therefore, it reads 8 bits by default with each read.



(c) The register that you are wanting to modify is at address `0x0024`, and its current value is `0b1100 1000`. On the diagram below, show signals present on the address, data, and control buses, when the `READ` operation is taking place.

Solution: The address bus will be set by the CPU to be equal to the address that is being read: `0x0024`, the control bus will indicate a `READ` command, which will ensure the data will be sent on the data bus: `0b1100 1000`



(d) Having read the byte, you want to force the LSB to be equal to 1. What logic operation will force a bit to be equal to 1? (*Hint: Looking at the complements within L4 may help*)

Solution: Taking a logic value a, it can be forced to 1 with the annihilation

property: $a + 1 = 1$.

(e) In pseudocode or C, what operation will result in the byte being modified to set the LSB to 1, while leaving the rest of the byte unmodified?

Solution: We know that we want to `OR` the LSB, with 1. But it is not possible to perform an operation on just a single bit, instead the entire word (in this case 8 bits) must be modified at the same time. Therefore, we need to determine values for the operands with the rest of the bits in the word we're modifying which will leave them unmodified.

Given that we are performing an `OR` operation, we can use the property that any value OR 0 will result in no change (identity property), or $b + 0 = b$. Therefore, we want to `OR` the LSB with 1, while we `OR` the other bits in the byte with 0:

So the full operation we want to perform is:

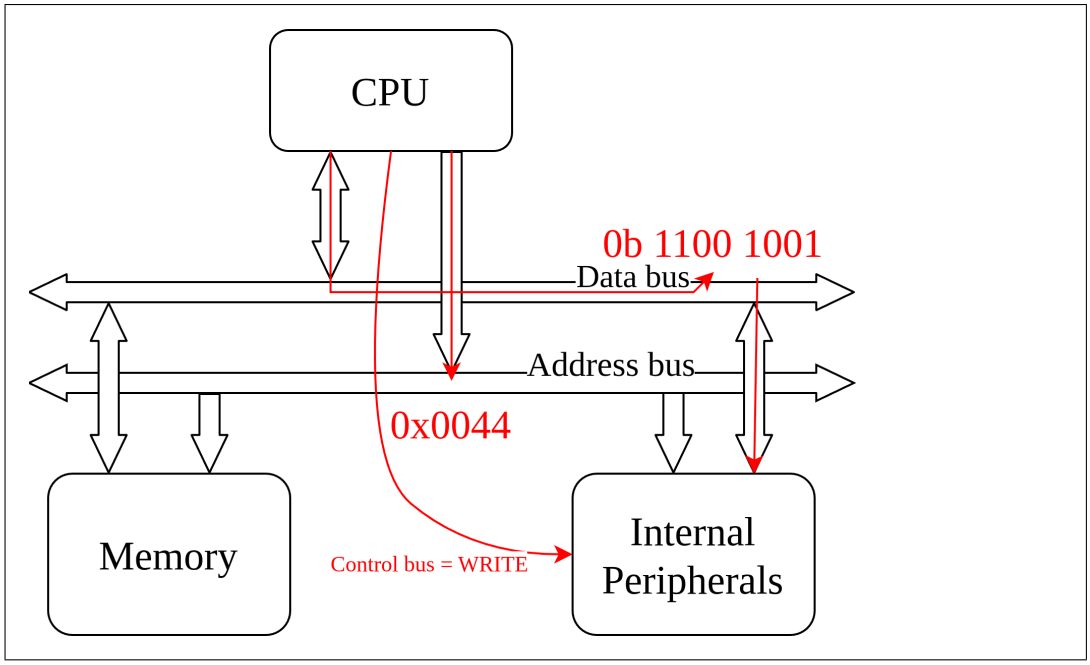
	1	1	0	0	1	0	0	0
`OR`	0	0	0	0	0	0	0	1
	1	1	0	0	1	0	0	1

Or, in C:

```
uint8_t updatedValue = currentValue | 0b00000001;
```

(f) Within which part of the computer architecture will this operation be performed, and how will the register be updated with the new value?

Solution: The operation will be performed within the CPU, specifically in the ALU. Therefore, the result has to leave the CPU and go back to the internal peripherals. The CPU will set the new value on the data bus, the address on the address bus (which confusingly uses a different address to perform a `WRITE (0x0044)` instruction that a `READ (0x0024)` instruction), and the control bus will indicate that it is performing a `WRITE` instruction:



(g) Extra for experts/curious only: What function does the above operation perform in the ATMega32u2? (*Hint: See section 15.9.1, p102 of the ATMega32u2 datasheet.*)

Q3.

Draw truth tables for each of the following logical functions of 3 Boolean variables:

(a) $z = f(a, b, c) = a + \overline{b} + \overline{c}$

Solution: There should be a 1 in any row where $a = 1$, as well as any row where $b = 0$ or where $c = 0$:

a	b	c	z
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

(b) $z = f(a, b, c) = \overline{a}(b + \overline{c})$

Solution:

a	b	c	z
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

When building a truth table like the one above, it can be helpful to add intermediate columns that show the truth values of individual parts of the overall Boolean expression:

a	b	c	$\overline{a}$	$\overline{c}$	$(b+\overline{c})$	z
0	0	0	1	1	1	1
0	0	1	1	0	0	0
0	1	0	1	1	1	1
0	1	1	1	0	1	1
1	0	0	0	1	1	0
1	0	1	0	0	0	0
1	1	0	0	1	1	0
1	1	1	0	0	1	0

(c) $z = f(a, b, c) = \overline{a}b\overline{c} + \overline{a}\overline{b}\overline{c} + \overline{a}bc$

Solution:

a	b	c	z
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Note that this is the same truth table for $z = \overline{a}(b + \overline{c})$.

$$\begin{aligned} z &= \overline{a}b\overline{c} + \overline{a}\overline{b}\overline{c} + \overline{a}bc \\ &= \overline{a}((b\overline{c}) + (\overline{b}\overline{c}) + (bc)) \\ &= \overline{a}((b + \overline{b})\overline{c} + (bc)) \\ &= \overline{a}(\overline{c} + (bc)) \\ &= \overline{a}((\overline{c} + b)(\overline{c} + c)) \\ &= \overline{a}(\overline{c} + b) \end{aligned}$$

(d) $g = f(p, t, u) = u + p\overline{t}$

Solution:

u	t	p	g
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

This answer is easy to check with a little Python:

```
>>> def bit_not(t):
...   return 1 if t == 0 else 0
...
>>> for u in (0,1):
...   for t in (0,1):
...     for p in (0,1):
...       g = u or (p and (bit_not(t)))
...       print("{} {} {} | {}".format(u, t, p, g))
...
0 0 0 | 0
0 0 1 | 1
0 1 0 | 0
0 1 1 | 0
1 0 0 | 1
1 0 1 | 1
1 1 0 | 1
1 1 1 | 1
```

Note that the built-in Python `and` and `or` operators work with 1 and 0, but that we need to define our own version of `not` to get a truth table made only of 1's and 0's.

Q4.

Apply the rules of Boolean algebra to simplify the expression:
$Q = \big(A \cdot \overline{B} \cdot C\big) + \big(A \cdot \big(\overline{B} \cdot \overline{C}\big)\big)$.

Solution:

$$\begin{aligned} Q &= \big(A \cdot \overline{B} \cdot C\big) + \big(A \cdot \big(\overline{B} \cdot \overline{C}\big)\big) \\ &= A \cdot \big(\overline{B} \cdot C + \overline{B} \cdot \overline{C}\big) \\ &= A \cdot \overline{B} \cdot \big(C + \overline{C}\big) \\ &= A \cdot \overline{B} \cdot (1) \\ &= A \cdot \overline{B} \end{aligned}$$

Q5.

Use truth tables to prove De Morgan's laws: $\overline{A} \cdot \overline{B} = \overline{A + B}$ and $\overline{A} + \overline{B} = \overline{A \cdot B}$.

Solution: Truth table for $\overline{A} \cdot \overline{B} = \overline{A + B}$:

A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$	$A + B$	$\overline{A + B}$
0	0	1	1	1	0	1
0	1	1	0	0	1	0
1	0	0	1	0	1	0
1	1	0	0	0	1	0

Since columns 5 and 7 match, we have $\overline{A} \cdot \overline{B} = \overline{A + B}$.

Similarly, for $\overline{A} + \overline{B} = \overline{A \cdot B}$:

A	B	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$	$A \cdot B$	$\overline{A \cdot B}$
0	0	1	1	1	0	1
0	1	1	0	1	0	1
1	0	0	1	1	0	1
1	1	0	0	0	1	0

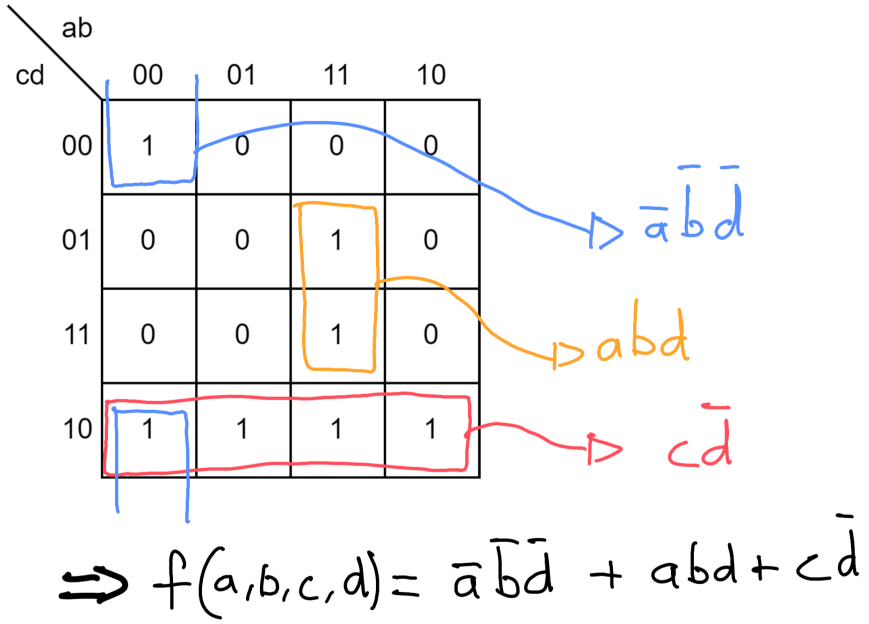
Again, columns 5 and 7 match, so $\overline{A} + \overline{B} = \overline{A \cdot B}$.

Q6.

Using a Karnaugh Map, simplify the following expression:

$$f(a,b,c,d) = \bar{a}\bar{b}\bar{c}\bar{d} + \bar{a}\bar{b}c\bar{d} + \bar{a}\bar{b}\bar{c}d + \bar{a}bc\bar{d} + \bar{a}bcd + a\bar{b}c\bar{d} + abc\bar{d}$$

Solution:



Q7.

Lecture 5 (CPU) tutorial questions

(a) Write an expression for a full adder's C_{out} in terms of the inputs A, B, and C_{in}.

Solution:

We are now just looking at the carry out from three input bits, so the behaviour is to determine whether there is a carry output from 3 input bits.

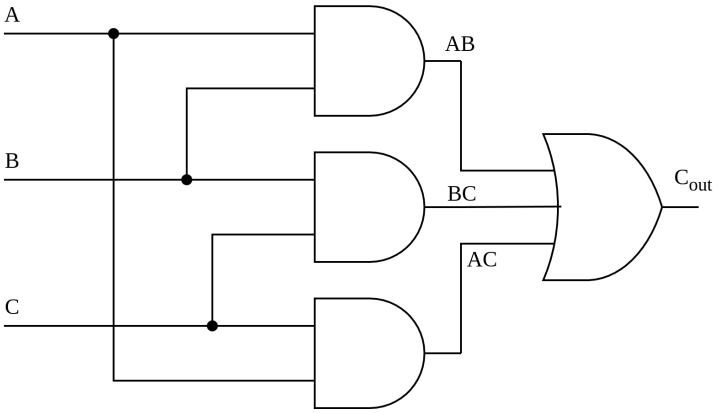
Remember, we are adding three inputs of equal significance, and have one output of the same significance as the input, and one of greater significance.

A	B	C_{in}	$A+B+C$	C_{out}
0	0	0	00	0
0	0	1	01	0
0	1	0	01	0
0	1	1	10	1
1	0	0	01	0
1	0	1	10	1
1	1	0	10	1
1	1	1	11	1

Thus, we can determine $C_{out} = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$. This simplifies to $C_{out} = AB + AC + BC$:

(b) Draw a circuit using AND, OR, and NOT logic gates that implements your expression for C_{out}.

Solution:



(c) Consider a comparator that operates on two two-bit numbers: $A = A_1A_0$, and $B = B_1B_0$:

(a) Write a logic expression for the *gt* output of this comparator. Your expression should use A_0, A_1, B_0, and B_1 as inputs.

Solution:

A_1A_0	B_1B_0	"Is A greater than B?"
00	00	0
00	01	0
00	10	0
00	11	0
01	00	1
01	01	0
01	10	0
01	11	0
10	00	1
10	01	1
10	10	0
10	11	0
11	00	1
11	01	1
11	10	1
11	11	0

TURN OVER

$gt = A_1\bar{A}_0\bar{B}_1\bar{B}_0 + \bar{A}_1A_0\bar{B}_1\bar{B}_0 + A_1\bar{A}_0\bar{B}_1B_0 + A_1A_0\bar{B}_1\bar{B}_0 + A_1A_0B_1\bar{B}_0 + A_1A_0\bar{B}_1B_0$

This is a fantastic place to use a K-map:

B_1B_0 \ A_1A_0	00	10	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

Which we can then determine $gt = A_1\bar{B}_1 + A_0\bar{B}_1\bar{B}_0 + A_1A_0\bar{B}_0$

(b) Draw a logic gate diagram based on your logic expression.

(c) Imagine how incredibly complex an 8-bit comparator would be using the same design process.

Q8.

For **Q1** (a) & (b), represent each of the functions using only `NAND` gates.

Solution:

$z = f(a, b, c) = a + \overline{b} + \overline{c}$
$= a + \overline{bc}$
$= a + d, where\ d = \overline{(b \cdot c)}$
$= a + d = \overline{\overline{a} \cdot \overline{d}}$
$\therefore z = \overline{\overline{(\overline{a \cdot a})} \cdot \overline{\overline{(\overline{b \cdot c})} \cdot \overline{(\overline{b \cdot c})}}}$

Solution:

$z = f(a, b, c) = \overline{a}(b + \overline{c})$

First, expand out the bracket using *distributive* properties:

$\overline{a} \cdot b + \overline{a} \cdot \overline{c}$

Implement `AND` with `(A NAND B) NAND (A NAND B)` (Not actually shown in full):
$\overline{\overline{\overline{a} \cdot b}} + \overline{\overline{\overline{a} \cdot \overline{c}}}$

Implement `OR` using `A OR B = NOT(A) NAND NOT(B)` (Again not actually showing the inversion in full):
$\overline{\overline{\overline{\overline{a} \cdot b}} \cdot \overline{\overline{\overline{a} \cdot \overline{c}}}}$

Finally, substitute NOT with NAND with itself:

$$\overline{\overline{(\overline{a \cdot a}) \cdot b} \cdot \overline{(\overline{a \cdot a}) \cdot (\overline{c \cdot c})}}$$

Unsimplified:

$$\overline{\overline{(\overline{a \cdot a}) \cdot b} \cdot \overline{(\overline{a \cdot a}) \cdot (\overline{c \cdot c})}}$$

Explore the simplified circuit here.

END OF QUESTIONS